

■ 2.7.2 Working with Names

In doing mathematical operations, the actual names that you give to particular symbols are essentially irrelevant. There are circumstances, however, when the form of names for symbols can be important. For example, you may want to find all symbols whose *names* start with the letter **x**.

If you type a particular symbol name, *Mathematica* will give you that symbol. Certain *Mathematica* functions, however, allow you to use character strings to refer to several different symbols at the same time. The character strings can contain special “metacharacters” which can stand for classes of other characters.

<code>?nameform</code>	give information on all objects whose names have a particular form
<code>Names["nameform"]</code>	give a list of symbol names that have a particular form
<code>Clear["nameform"]</code>	clear the values of all symbols whose names have a particular form

Functions that work with names.

This prints a list of all objects whose names begin with **B**.

```
In[1]:= ?B*
BaseForm      BesselY      Block
Begin         Beta       Bottom
BeginPackage  Binomial    BoxRatios
Below         Blank       Boxed
BernoulliB    BlankForm   Break
BesselI       BlankNullSequence Byte
BesselJ       BlankSequence ByteCount
BesselK
```

Here is a *Mathematica* list of symbols whose names begin with **V**. The entries in this list are all character strings.

```
In[1]:= Names["V*"]
Out[1]= {ValueForm, ValueList, Variables, VectorQ,
        VerticalForm, ViewPoint}
```

This clears the values of all symbols whose names begin with **x**. Since there are no symbols with such names in the current *Mathematica* session, this does nothing here.

```
In[2]:= Clear["x*"]
```

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

<code>*</code>	zero or more alphanumeric characters
<code>**</code>	zero or more alphanumeric characters, or <code>\$</code>
<code>@</code>	one or more lower-case letters
<code>*</code> etc.	literal <code>*</code> etc.

Metacharacters used in specifying names.

You can use metacharacters to give “patterns” for strings. You should realize, however, that these patterns have nothing to do with the kind of patterns for expressions that we discussed in Sections 2.2 and 2.3. They relate only to the textual form of strings.

```
StringMatchQ["string", "pattern"]
```

yield **True** if *string* matches the textual pattern given,
and **False** otherwise

A function for textual pattern matching.

You can use metacharacters to build up “regular expressions” that stand for classes of strings. The regular expression `a*b`, for example, stands for any string that begins with `a`, ends with `b`, and has any number of other characters in between. `a*b*` stands for strings that start with `a`, then have any number of other characters, including at least one `b`.

The metacharacters provided in *Mathematica* include `@`, which stands for any sequence of lower-case letters. This metacharacter is useful when you want to work on user-defined symbols, whose names contain no upper-case letters.

In this case, the string matches the pattern `In[3]:= StringMatchQ["aaaabbcbbb", "a*b*"]`
you have given. `Out[3]= True`

As well as recognizing symbols whose names have a particular form, you may sometimes want to *generate* automatically symbols with various different names. Given a character string such as `"aaa"`, you can generate a symbol with that name using `ToExpression["aaa"]`.

Sometimes you may need to produce a symbol whose name is guaranteed not to have been used before. You can do this using the function `Unique`.

<code>Unique["sss"]</code>	generate a symbol with a name of the form <code>sssnnn</code> , that has not been used before
----------------------------	---

A function for generating symbols with unique names.

`Unique` generates a unique new symbol every time it is called.

```
In[4]:= {Unique["a"], Unique["a"], Unique["b"]}
Out[4]= {a1, a2, b1}
```

You may sometimes need to use `Unique` to get names for “dummy variables” that you can be sure will not conflict with other variables. You should use `Unique` sparingly, however; if at all possible, call your variables `a[1]`, `a[2]`, etc., rather than `a1`, `a2`, etc.